WILDTRAIL

Safari Booking System

ONE WEB APPLICATION · SIX CONNECTED MODULES

SE1020 • Object Oriented Programming

Group 26-MTR-Y1S2-WD-009



Six contributors, one project

#	Student	Member	Component	Main demonstration
01	IT26100894	Mekhala B.G.R.S	Users and customers	Register, profile, staff access
02	IT26100895	Mihiran S.P.D	Safaris and packages	Catalogue, price, package CRUD
03	IT26100896	Nayanajith W.R.S.C	Bookings	Availability, change, cancel
04	IT26100899	Senevirathna M.K.K.T	Guides and vehicles	Schedules, guide file CRUD
05	IT26100900	Lakshani W.G.C	Payments	Pending, cash, receipt, refund
06	IT26100901	Gamage C.I.P	Reviews	Submit, moderate, edit, delete

Each row leads to its classes, pages and tests in README → Complete project map.

One customer journey crosses all six modules

01 Customer signs in → 02 Chooses a safari → 03 Books a date →

04 Gets vehicle/guide → 05 Pays on arrival → 06 Leaves a review ✓

Controller → service/model → repository. Business rules live in Java, not only in the browser.

Java classes connected across the application

01 User

User → Customer / Admin
AuthService

02 SafariPackage

Seven safari subtypes
PackageService

04 Vehicle / Guide

Jeep subtypes · NaturalistGuide
VehicleService

03 Booking

BookingService · Booking
JdbcBookingRepository

05 Payment

PaymentProcessor → Payment
CashPayment / CardPayment

06 Review

Review → VerifiedReview
ReviewService

Users, identity and customer records

User

abstract model

– int id
– String email, fullName

+ getRole(): String
+ canManageSystem(): boolean

AuthService

service

– JdbcUserRepository users
– BCryptPasswordEncoder encoder

+ login(...)
+ register(...)

CustomerService

service

– JdbcCustomerRepository people

+ updateCustomer(...)
+ deleteCustomer(...)

Inheritance: Customer and Admin extend User · Controllers call services · repositories persist records.

Customer

extends User

Admin

extends User

AuthController

→ AuthService

ProfileController

→ session + JDBC

CustomerController

→ CustomerService

AdminAccountController

→ AuthService

JdbcUserRepository

user account SQL

JdbcCustomerRepository

customer record SQL

Safari package class hierarchy

SafariPackage

abstract model

− double basePrice
− int maxGuests, durationHours

+ calculatePrice(int): double
+ validateGuests(int): void

PackageService

service

− JdbcPackageRepository catalog

+ packages(...)
+ createPackage(...)

JdbcPackageRepository

JDBC repository

− JdbcTemplate jdbc

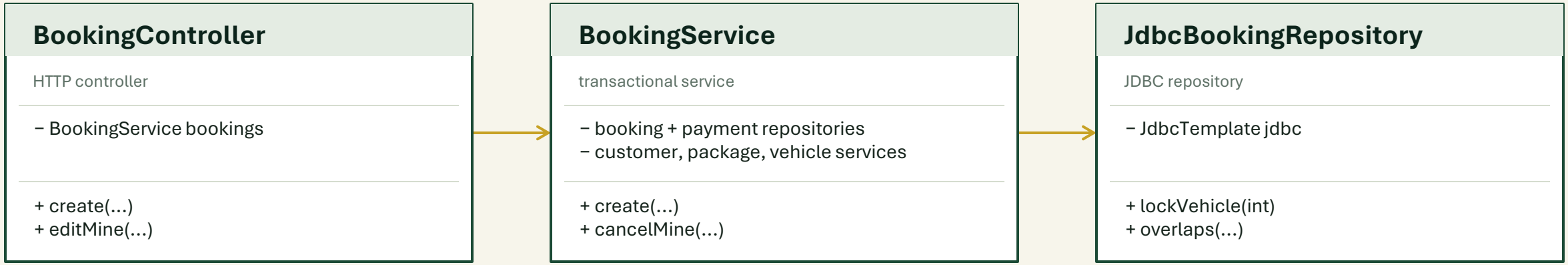
+ findPackage(int)
+ updatePackage(...)

Polymorphism: concrete safari types provide type details; some override calculatePrice().

MorningSafari	EveningSafari	HalfDaySafari	FullDaySafari
extends SafariPackage	extends SafariPackage	extends SafariPackage	extends SafariPackage

PrivateSafari	FamilySafari	PhotographySafari	PackageController
extends SafariPackage	extends SafariPackage	extends SafariPackage	→ PackageService

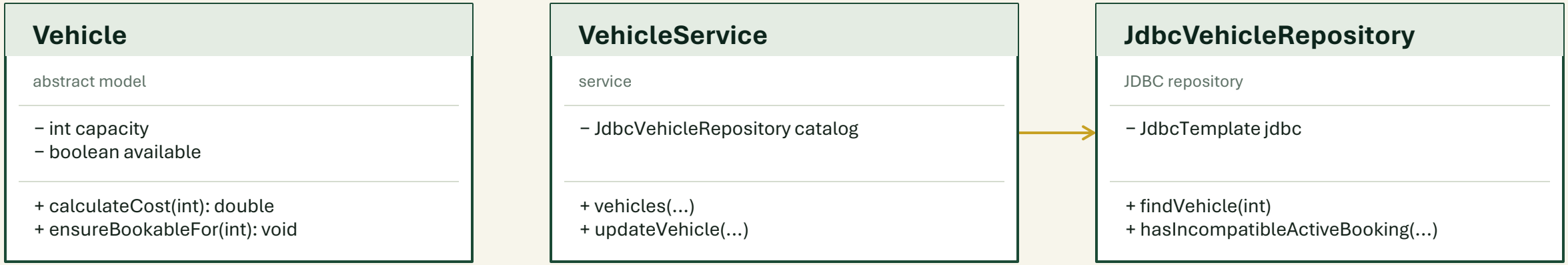
Booking lifecycle and availability



Booking.quote() combines package and vehicle prices; the service checks conflicts before saving.

Booking status transition model	**PackageService** cross-module dependency	**VehicleService** cross-module dependency	**JdbcPaymentRepository** cross-module dependency
JdbcCustomerRepository cross-module dependency	**PaymentFactory** creates payment model	**SessionUser** customer identity	**SafariPackage / Vehicle** price and capacity

Vehicle and guide class hierarchy



Inheritance defines vehicle types; guide records use a file, while vehicles use SQL.

SafariJeep extends Vehicle	**HiluxSafariJeep** extends SafariJeep	**FamilySafariJeep** extends SafariJeep	**LuxuryJeep** extends Vehicle	**Guide** guide model
NaturalistGuide extends Guide	**FileGuideRepository** guide file CRUD	**GuideController** guide endpoints	**VehicleController** vehicle endpoints	**TeamController** guide/team view

Payment types and receipt flow

PaymentProcessor

interface

+ process(): PaymentResult

Payment

abstract model

− double amount

+ getAmount(): double
getMethod(): String

PaymentService

service

− JdbcBookingRepository bookings
− JdbcPaymentRepository payments

+ updatePaymentStatus(...)
+ deleteFailedPayment(int)

Payment implements PaymentProcessor; the demo card option remains pending and charges nothing.

CashPayment

extends Payment

CardPayment

extends Payment

DemoCardPayment

extends Payment

PaymentFactory

creates payment subtype

PaymentResult

record: status + amount

JdbcPaymentRepository

payment SQL

PaymentController

staff payment endpoints

ReceiptController

receipt endpoint

Review model and moderation

Review

validated model

− int rating
− String comment

+ getRating(): int
+ getType(): String

ReviewService

service

− JdbcReviewRepository people
− JdbcCustomerRepository customers

+ submitReview(...)
+ approveReview(...)

JdbcReviewRepository

JDBC repository

− JdbcTemplate jdbc

+ createReview(...)
+ setReviewApproved(...)

A review has validation, a verified subtype, moderation, and persistent records.

VerifiedReview

extends Review

ReviewController

→ ReviewService

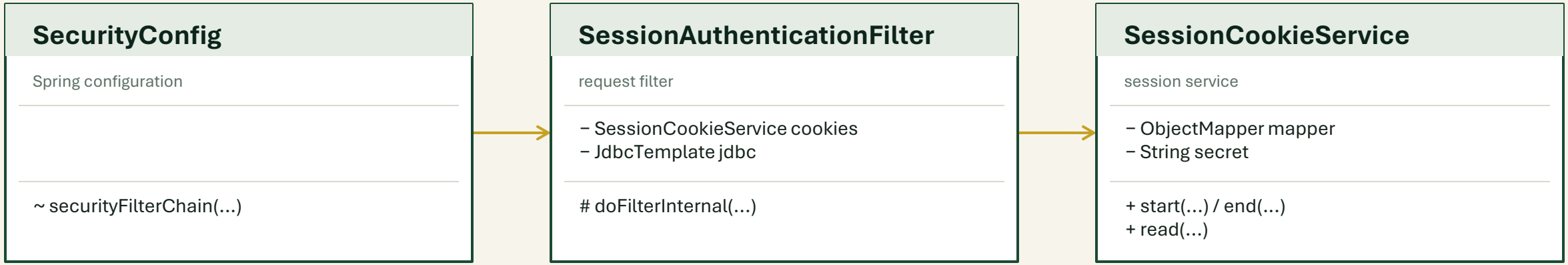
MyReviewsController

customer review view

Customer / Booking

ownership and eligibility

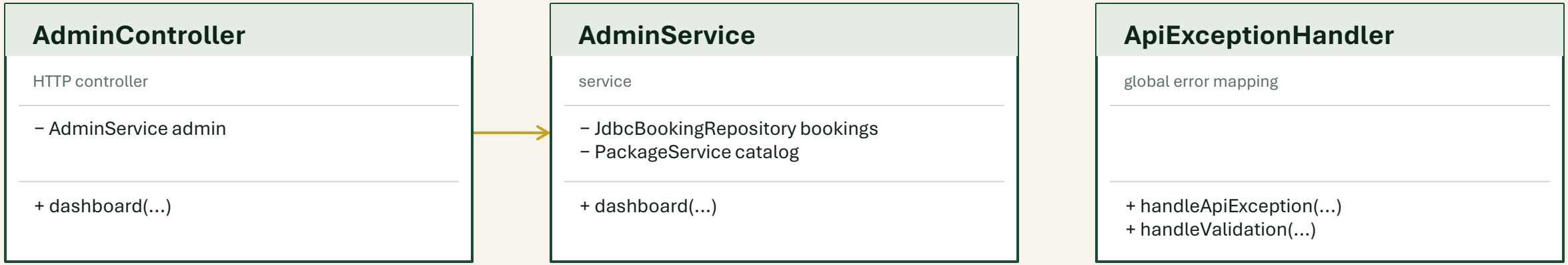
Security and session classes



Requests pass session and origin checks before protected controllers run.

SessionUser authenticated identity	**OriginGuard** allowed origins	**LoginThrottle** rate limits	**AuditFilter** audit log
AdminBootstrap initial staff account	**DatabaseSeeder** catalogue seed	**WebConfig** CORS configuration	

Shared API and error handling



Shared controllers use the same services, validation and error response policy.

ContactController	**InboxController**	**FileReportController**	**HealthController**	**SessionController**
contact form	inbox	text report	health endpoint	current session
ApiException	**JdbcInsert**	**RequestValues**	**WildTrailApplication**	
status + message	insert helper	request validation	Spring Boot entry point	

Website and backend integration



The website is one application: client and staff pages use the Java API and shared stores.

BookingWizard.tsx booking + payment UI	**Navbar.tsx** public navigation	**AdminSidebar.tsx** staff navigation	**src/app/admin/(panel)** staff pages
src/lib/api.ts typed HTTP client	**src/lib/session.ts** browser session state	**H2 SQL** main data store	**resources.txt** guide file store

Every component has a clear data path

Component	Create / Read / Update / Delete example	Primary store
Users	Register · profile · remove eligible account	H2 SQL
Safaris	Add · filter · edit · remove unused package	H2 SQL
Bookings	Reserve · history · change · cancel	H2 SQL
Guides / vehicles	Add · schedule · edit · remove unused	Guide file + H2 SQL
Payments	Pending · receipt · status · failed removal	H2 SQL
Reviews	Submit · view · edit · moderate/delete	H2 SQL

Guide file: resources.txt • Payment card option: simulation only; no card data or charge.

Four concepts visible in the Java code

Encapsulation

Booking guards status, dates and price.

Inheritance

SafariPackage → concrete safari types.

Polymorphism

calculatePrice() varies by subtype.

Abstraction

PaymentProcessor defines processing.

A viva answer should name a class, a method and the behavior it changes.

A single demo through the whole application

01 Browse packages and destinations

02 Register and create a booking

03 Show an eligible change or cancellation

04 Inspect guide / vehicle schedule

05 Record cash collection and receipt

06 Submit and moderate a review

24-hour notice and availability checks are enforced by the Java API.

Tests, Git history and project documents

57
Java tests

11
Browser flows

4
Access checks

12
Viewport widths

Individual review trail

#14 Mekhala #12 Mihiran #7/#9 Nayanajith
#16 Senevirathna #10 Lakshani #11 Gamage

Six short answers to remember

Where are rules?

Java service and model.

Where is data?

H2 SQL; guides in resources.txt.

How is staff access checked?

Server-side session and role.

How are booking conflicts stopped?

Transactions, locks and validation.

Does card payment charge?

No. Demo only, pending.

Why 24 hours?

Notice protects planned resources.